

Stepsperepoch:- The number of samples that are run before declaring one epoch as finished and starting the next epoch.

RECOMMENDED PRACTICES

Before diving into the deep end of functional learning and APIs, keeping some well-known principles in check are important to help ease out any coding chinks or output errors:-

Using consistent variable names:- A rather simple practice. If all you feed in the garbage into the system, all you're going to get is garbage. Maintain a consistent scheme for the variables, outputs and hidden layers.

Beware the numbers:- Lies, Damned Lies and statistics- the three most common lies as per the former US President. Don't be too focused on the accuracies that the API produces. Split the data in an even scheme using cross-validation or normal percentage splits. Test out the model statistics using validation techniques like ROC and AUC curves-Receiver Operating Curve, Area Under Curve.

Review the layers:- Always print out the summaries of the model including the hidden layers to ensure that the model works as intended.

Separate the sub-models:- Consider separating the development of sub-models and combine the sub-models at the end.

CONCLUSION

Functional APIs barely scratch the surface to the myriad of possibilities that modern tools provide with a little bit of coding expertise. Whatever be the need, test through the waters by implementing a small model on keras using the python or tensorflow environment. Keep a different split for the testing and training data depending upon the nature of the project and shape the systems with different parameters in the code.

Always remember to have a standard case with no special model definitions. This will work as the controlled example showing the minimum accuracy that the functional API can get you. Model building for API's is finally a question of what the data looks like and what the output is. This is a good time to bring up the need for exploratory analysis. Fire up a PCA-Principal Component Analysis, scatter plots, bar charts and understand the distribution of the data.

After interpreting what works and what doesn't selectively remove the inputs that have no weight. Refine. Rebuild. Test. Validate. Repeat. That's what it takes to run a successful API. 